

LAPORAN PROPOSAL SKRIPSI

Analisis Perbandingan Performa Framework Frontend VueJS dan Svelte pada Prototipe Website Listing Properti Berdasarkan Studi Kasus Airbnb Berdasarkan Segi Komputasional dan Pengalaman Pengguna



Disusun oleh:

**Exel Boy Alfanso
NIM: 235150201111019**

**PROGRAM STUDI TEKNIK INFORMATIKA
DEPARTEMEN TEKNIK INFORMATIKA
FAKULTAS ILMU KOMPUTER
UNIVERSITAS BRAWIJAYA
MALANG
2026**

DAFTAR ISI

1	PENDAHULUAN	1
1.1	Identitas Mahasiswa	1
1.2	Keminatan dan Bidang Penelitian	1
1.3	Jenis Penelitian	1
1.4	Judul / Topik Skripsi	1
1.5	Latar Belakang	1
1.6	Rumusan Masalah	3
1.7	Tujuan	3
1.8	Batasan Masalah	3
2	LANDASAN KEPUSTAKAAN	5
2.1	Website Responsif	5
2.2	Framework Frontend	5
2.3	Vue JS	5
2.4	Svelte	5
2.5	Core Web Vitals	6
2.5.1	Largest Contentful Paint	6
2.5.2	Interaction to Next Paint	6
2.5.3	Cumulative Layout Shift	6
2.6	Efisiensi Sumber Daya	7
2.7	ElysiaJS dan Eden Treaty	7
2.8	Website Listing Properti	7
2.8.1	Unit Testing	7
2.8.2	Integration Testing	8
2.8.3	End-to-End Testing	8
2.8.4	Web Performance Testing	8
2.8.5	User Experience	9
2.8.6	Usability Testing	9
2.8.7	System Usability Scale (SUS)	9
2.8.8	Vitest	10
2.8.9	Playwright	10
2.8.10	Google Lighthouse	10
2.8.11	Chrome DevTools	11
3	METODOLOGI PENELITIAN	12
3.1	Rancangan Penelitian	12
3.2	Persiapan dan Pengembangan Prototipe	13
3.3	Validasi Kesetaraan Fitur	14
3.3.1	Unit Testing	14

3.3.2	Integration Testing	15
3.3.3	End-to-End Testing	15
3.4	Pengujian Performa Komputasional	15
3.4.1	Lingkungan Pengujian	15
3.4.2	Pengujian LCP dan CLS	16
3.4.3	Pengujian Interaction to Next Paint	16
3.4.4	Pengujian Efisiensi Sumber Daya	18
3.5	Evaluasi User Experience dan Usability	18
3.5.1	Partisipan Penelitian	18
3.5.2	Skenario Pengujian	18
3.5.3	Prosedur Pengujian	19
3.5.4	User Experience Questionnaire	19
3.5.5	System Usability Scale	19
3.6	Teknik Analisis Data	19
3.6.1	Analisis Performa Komputasional	19
3.6.2	Analisis UEQ	20
3.6.3	Analisis SUS	20
3.7	Analisis Komparatif Hasil Pengujian	20

DAFTAR REFERENSI

21

DAFTAR GAMBAR

3.1	Flowchart Tahapan Penelitian	13
-----	------------------------------------	----

DAFTAR TABEL

2.1	Threshold Core Web Vitals	7
3.1	Variabel Kontrol Penelitian	14
3.2	Skenario Pengujian Kesetaraan Fitur	15
3.3	Konfigurasi Lingkungan Pengujian	16
3.4	Skenario Interaksi Pengukuran INP	18
3.5	Threshold Core Web Vitals	19
3.6	Instrumen dan Alat Pengukuran	20

BAB 1

PENDAHULUAN

1.1 Identitas Mahasiswa

- Nama: Exel Boy Alfanso
- NIM: 235150201111019
- Program Studi: Teknik Informatika
- Departemen: Teknik Informatika

1.2 Keminatan dan Bidang Penelitian

Pada skripsi ini, keminatan saya adalah MGM (Multimedia, Game, dan Mobile) dan spesifik Bidang penelitian yang diambil khususnya pada kajian pengembangan aplikasi web, performa antarmuka web responsif, serta evaluasi framework frontend modern pada perangkat bergerak.

1.3 Jenis Penelitian

Jenis penelitian ini adalah penelitian implementatif dengan pendekatan pengembangan. Penelitian ini menghasilkan artefak berupa prototipe mobile web listing properti yang dikembangkan dalam dua versi frontend, yaitu VueJS dan Svelte. Kedua frontend menggunakan service backend yang sama berbasis ElysiaJS dengan Eden Treaty sebagai type-safe API client. Setelah prototipe dikembangkan, dilakukan pengujian dan analisis perbandingan performa menggunakan Core Web Vitals dan metrik efisiensi sumber daya.

1.4 Judul / Topik Skripsi

Judul/topik yang akan saya bawa pada skripsi ini adalah Analisis Perbandingan Performa Framework Frontend VueJS dan Svelte Menggunakan Core Web Vitals dan Efisiensi Sumber Daya pada Prototipe Mobile Web Listing Properti Berbasis Studi Kasus Airbnb dengan Service Backend ElysiaJS dan Eden Treaty.

1.5 Latar Belakang

Perkembangan aplikasi web modern menuntut website untuk memiliki performa yang cepat, responsif, dan efisien dalam penggunaan sumber daya. Kebutuhan tersebut menjadi semakin penting pada website berbasis listing yang menampilkan banyak konten visual dan elemen antarmuka secara bersamaan. Pada website listing properti, misalnya, halaman dapat memuat gambar properti, informasi lokasi, harga, rating, serta berbagai komponen interaktif seperti pencarian, penyaringan, pengurutan, dan pemuatan data tambahan. Karakteristik tersebut menyebabkan sisi frontend memiliki peran

penting terhadap waktu pemuatan halaman, respons terhadap interaksi pengguna, stabilitas tampilan, serta jumlah sumber daya yang perlu diproses oleh browser.

VueJS dan Svelte merupakan framework frontend berbasis komponen yang dapat digunakan untuk membangun antarmuka web interaktif. Meskipun memiliki tujuan yang serupa, keduanya menggunakan pendekatan berbeda dalam proses pembaruan antarmuka. VueJS menggunakan sistem reaktivitas yang bekerja bersama mekanisme Virtual DOM pada saat aplikasi berjalan, sedangkan Svelte menggunakan pendekatan berbasis compiler dengan mengompilasi komponen menjadi kode JavaScript pada proses build. Perbedaan pendekatan tersebut berpotensi menghasilkan karakteristik performa yang berbeda, baik dari sisi proses rendering, jumlah JavaScript yang dikirimkan ke browser, penggunaan sumber daya, maupun respons aplikasi ketika terjadi perubahan data.

Penelitian sebelumnya telah menunjukkan bahwa perbedaan strategi rendering pada framework frontend dapat menghasilkan karakteristik performa yang berbeda. (Ollila, Mäkitalo and Mikkonen, 2022) membandingkan strategi rendering Angular, React, Vue, Svelte, dan Blazor menggunakan sejumlah benchmark untuk mengukur perubahan biaya rendering seiring meningkatnya kompleksitas aplikasi. Penelitian tersebut menunjukkan adanya perbedaan pola performa berdasarkan strategi rendering yang digunakan oleh masing-masing framework. Selain itu, (Putra et al., 2025) membandingkan Svelte, React, dan Vue menggunakan aplikasi web sederhana yang dibuat secara identik dengan pengukuran berupa rendering time, bundle size, penggunaan memori saat idle, dan efisiensi operasi DOM. Penelitian tersebut menunjukkan bahwa perbedaan arsitektur framework dapat memengaruhi performa teknis aplikasi.

Meskipun demikian, penelitian terdahulu tersebut masih berfokus pada benchmark rendering atau implementasi aplikasi yang relatif sederhana. Kondisi tersebut belum sepenuhnya menggambarkan karakteristik aplikasi web yang memiliki banyak konten visual serta melibatkan beberapa bentuk interaksi dan perubahan data dalam satu aplikasi. Oleh karena itu, masih diperlukan kajian mengenai bagaimana perbedaan pendekatan VueJS dan Svelte menghasilkan karakteristik performa ketika diterapkan pada prototipe aplikasi yang memiliki workload lebih representatif, seperti website listing properti yang mencakup pemuatan sejumlah gambar dan komponen berulang, pencarian, penyaringan, pengurutan, pemuatan data tambahan, dan navigasi menuju detail properti. Selain itu, evaluasi yang menggabungkan metrik performa yang berorientasi pada pengalaman pengguna melalui Core Web Vitals dengan efisiensi sumber daya dan respons interaksi dapat memberikan gambaran yang lebih menyeluruh mengenai perbedaan performa kedua framework.

Berdasarkan kesenjangan tersebut, penelitian ini mengembangkan dua prototipe website listing properti menggunakan VueJS dan Svelte dengan fitur dan perilaku yang dibuat setara. Kedua implementasi menggunakan desain, dataset, aset, struktur konten, dan service backend yang sama. ElysiaJS digunakan sebagai service backend untuk menyediakan data melalui API, sedangkan Eden Treaty digunakan untuk mendukung komunikasi API yang type-safe antara frontend dan backend. Kesetaraan tersebut diterapkan agar perbedaan hasil pengujian dapat lebih difokuskan pada karakteristik implementasi frontend masing-masing framework, bukan akibat perbedaan data, desain, fitur, atau layanan backend.

Prototipe yang dikembangkan mengacu pada karakteristik umum website listing properti modern, khususnya pola penyajian daftar properti yang kaya konten visual serta mendukung fitur seperti pencarian, penyaringan, pengurutan, halaman detail, dan pemuatan data secara bertahap. Airbnb dapat digunakan sebagai salah satu referensi karak-

teristik antarmuka dan pola interaksi pada domain tersebut. Namun, penelitian ini tidak bertujuan untuk mereplikasi Airbnb secara penuh maupun mengevaluasi teknologi yang digunakan oleh Airbnb, melainkan menggunakan karakteristik website listing properti sebagai konteks untuk melakukan perbandingan performa VueJS dan Svelte.

1.6 Rumusan Masalah

1. Bagaimana perbedaan performa komputasional antara VueJS dan Svelte pada prototipe website listing properti berdasarkan Core Web Vitals, efisiensi sumber daya, dan respons interaksi pengguna?
2. Bagaimana perbandingan User Experience pada prototipe website listing properti yang dikembangkan menggunakan VueJS dan Svelte berdasarkan hasil User Experience Questionnaire (UEQ)?
3. Bagaimana perbandingan usability pada prototipe website listing properti yang dikembangkan menggunakan VueJS dan Svelte berdasarkan hasil System Usability Scale (SUS)?
4. Bagaimana hasil perbandingan performa komputasional, User Experience, dan usability antara VueJS dan Svelte dapat menjadi pertimbangan dalam pemilihan framework frontend untuk pengembangan website listing yang kaya konten visual dan memiliki fitur interaktif?

1.7 Tujuan

1. Menganalisis perbedaan performa komputasional antara VueJS dan Svelte pada prototipe website listing properti berdasarkan Core Web Vitals, efisiensi sumber daya, dan respons interaksi pengguna.
2. Menganalisis perbandingan User Experience pada prototipe website listing properti yang dikembangkan menggunakan VueJS dan Svelte berdasarkan hasil User Experience Questionnaire (UEQ).
3. Menganalisis perbandingan usability pada prototipe website listing properti yang dikembangkan menggunakan VueJS dan Svelte berdasarkan hasil System Usability Scale (SUS).
4. Menganalisis hasil perbandingan performa komputasional, User Experience, dan usability antara VueJS dan Svelte sebagai pertimbangan dalam pemilihan framework frontend untuk pengembangan website listing yang kaya konten visual dan memiliki fitur interaktif.

1.8 Batasan Masalah

1. Penelitian hanya membandingkan implementasi frontend menggunakan VueJS dan Svelte. ElysiaJS digunakan sebagai service backend yang sama pada kedua prototipe dan tidak menjadi objek utama perbandingan.
2. Prototipe yang dikembangkan dibatasi pada fitur utama website listing properti, yaitu halaman daftar properti, halaman detail properti, pencarian, penyaringan (filter), pengurutan (sorting), pagination atau load more, serta wishlist sederhana.

3. Airbnb digunakan sebagai referensi karakteristik antarmuka dan pola interaksi website listing properti, bukan sebagai objek yang direplikasi secara penuh maupun sebagai objek evaluasi teknologi yang digunakan oleh Airbnb.
4. Data properti, lokasi, tipe properti, fasilitas, rating, host, dan gambar yang digunakan pada kedua prototipe merupakan data dummy atau aset bebas lisensi yang dibuat sama untuk kedua implementasi.
5. Penelitian tidak mencakup fitur authentication, authorization, pembayaran, transaksi pemesanan sebenarnya, chat, host dashboard, admin dashboard, sistem kalender real-time, dan integrasi peta sebenarnya.
6. Pengujian performa komputasional dilakukan pada production build dengan kondisi perangkat, browser, viewport, dataset, aset, fitur, dan skenario pengujian yang dibuat sama untuk implementasi VueJS dan Svelte.
7. Pengukuran performa komputasional dibatasi pada Core Web Vitals yang terdiri atas Largest Contentful Paint (LCP), Interaction to Next Paint (INP), dan Cumulative Layout Shift (CLS), serta metrik efisiensi sumber daya dan respons interaksi yang meliputi ukuran JavaScript, total transferred size, request count, dan waktu respons pada skenario interaksi yang telah ditentukan.
8. Evaluasi User Experience dan usability dilakukan terhadap kedua prototipe menggunakan skenario penggunaan yang sama. User Experience diukur menggunakan User Experience Questionnaire (UEQ), sedangkan usability diukur menggunakan System Usability Scale (SUS).
9. Penelitian berfokus pada perbandingan VueJS dan Svelte dalam konteks prototipe website listing properti yang dikembangkan. Hasil penelitian tidak ditujukan untuk menentukan framework terbaik secara umum maupun digeneralisasikan terhadap seluruh jenis aplikasi web.

BAB 2

LANDASAN KEPUSTAKAAN

2.1 Website Responsif

Website responsif merupakan website yang dirancang agar tampilan dan tata letaknya dapat menyesuaikan berbagai ukuran layar, termasuk desktop, tablet, dan mobile. Pada tampilan mobile, website responsif perlu memperhatikan penyusunan konten, ukuran elemen interaktif, keterbacaan teks, navigasi, serta efisiensi resource agar tetap nyaman digunakan. Dalam penelitian ini, responsivitas mobile menjadi konteks pengujian karena prototipe listing properti akan diuji pada viewport mobile.

2.2 Framework Frontend

Framework frontend merupakan kerangka kerja yang digunakan untuk membangun antarmuka pengguna pada aplikasi web. Framework frontend membantu pengembangan dalam menyusun komponen, mengelola struktur aplikasi, mengatur interaksi pengguna, dan membangun tampilan web modern. (Ferreira, Borges and Valente, 2022) menjelaskan bahwa framework frontend JavaScript seperti Vue, React, Angular, Ember, dan Backbone digunakan untuk membangun antarmuka web modern berbasis komponen. Selain itu, (Sofi'ie and Qoiriah, 2023) menyatakan bahwa framework frontend menjadi komponen penting dalam pengembangan website karena dapat membantu proses pengembangan, manajemen kode, performa, dan konsistensi aplikasi.

2.3 Vue JS

Vue.js adalah framework frontend JavaScript yang digunakan untuk membangun antarmuka web berbasis komponen. (Ferreira, Borges and Valente, 2022) menjelaskan bahwa Vue merupakan progressive framework yang menggunakan pola Model-View-View Model (MVVM) dan mendukung mekanisme one-way maupun two-way data binding. Dalam penelitian lain, (Sofi'ie and Qoiriah, 2023) menyebutkan bahwa Vue memiliki karakteristik struktur aplikasi yang sederhana dan kemampuan pengelolaan data yang mudah. Pada penelitian ini, Vue.js digunakan sebagai salah satu framework frontend yang dibandingkan dalam pengembangan prototipe website responsif listing properti.

2.4 Svelte

Svelte adalah framework frontend berbasis komponen yang menggunakan pendekatan compiler-based. Berbeda dengan framework yang membawa runtime besar ke browser, Svelte mengkompilasi komponen menjadi JavaScript pada saat proses build. Dengan pendekatan ini, Svelte dapat menghasilkan kode yang lebih spesifik terhadap kebutuhan komponen dan berpotensi mengurangi beban runtime pada browser. Karakteristik ini relevan untuk website responsif yang memiliki banyak komponen berulang dan membutuhkan antarmuka yang tetap responsif pada tampilan mobile.

2.5 Core Web Vitals

Core Web Vitals merupakan kumpulan metrik yang digunakan untuk mengukur kualitas pengalaman pengguna pada website. Metrik ini mencakup Largest Contentful Paint, Interaction to Next Paint, dan Cumulative Layout Shift. LCP digunakan untuk mengukur waktu munculnya konten utama, INP digunakan untuk mengukur respons interaksi, dan CLS digunakan untuk mengukur stabilitas visual halaman. Dalam penelitian ini, Core Web Vitals digunakan sebagai metrik utama untuk membandingkan performa prototipe VueJS dan Svelte. (Andriyanai et al., 2025)

2.5.1 Largest Contentful Paint

Largest Contentful Paint (LCP) merupakan metrik yang mengukur waktu yang dibutuhkan hingga elemen konten terbesar pada area tampilan (viewport) berhasil ditampilkan kepada pengguna. Elemen yang diperhitungkan dapat berupa gambar, elemen gambar yang terdapat di dalam SVG, gambar latar tertentu, video, maupun blok teks yang berukuran besar. Dengan demikian, LCP digunakan sebagai indikator untuk mengetahui seberapa cepat konten utama suatu halaman dapat dilihat oleh pengguna.

2.5.2 Interaction to Next Paint

Interaction to Next Paint (INP) merupakan metrik Core Web Vitals yang digunakan untuk mengukur tingkat responsivitas suatu halaman terhadap interaksi pengguna. INP mengamati latensi interaksi seperti klik, sentuhan, dan input menggunakan papan ketik selama pengguna mengakses halaman. Nilai INP merepresentasikan waktu antara terjadinya interaksi pengguna hingga browser dapat menampilkan pembaruan visual berikutnya sebagai respons terhadap interaksi tersebut.

Berbeda dengan metrik yang hanya mengukur interaksi pertama, INP mempertimbangkan berbagai interaksi yang terjadi sepanjang penggunaan halaman dan menggunakan interaksi dengan latensi yang paling tinggi dengan mekanisme tertentu untuk mengabaikan outlier. Nilai INP yang rendah menunjukkan bahwa halaman mampu memberikan umpan balik visual secara konsisten dalam waktu yang singkat. Google menetapkan INP sebesar 200 milidetik atau kurang sebagai kategori baik, sedangkan nilai lebih dari 500 milidetik dikategorikan buruk.

2.5.3 Cumulative Layout Shift

Cumulative Layout Shift (CLS) merupakan metrik yang mengukur stabilitas visual halaman berdasarkan perubahan posisi elemen yang terjadi secara tidak terduga selama halaman digunakan. Perubahan tata letak dapat terjadi ketika suatu elemen berpindah setelah elemen lain selesai dimuat, misalnya gambar yang tidak memiliki dimensi yang ditentukan sebelumnya atau konten yang dimasukkan secara dinamis. CLS menghitung kelompok perubahan tata letak tidak terduga yang paling signifikan selama siklus hidup halaman.

Berbeda dengan LCP dan INP yang menggunakan satuan waktu, CLS merupakan nilai tanpa satuan. Nilai yang semakin mendekati nol menunjukkan bahwa tampilan halaman semakin stabil. Google menetapkan nilai CLS sebesar 0,1 atau kurang sebagai kategori baik. Stabilitas visual penting karena perubahan posisi elemen secara tiba-tiba dapat

mengganggu pengguna ketika membaca konten atau berinteraksi dengan suatu elemen antarmuka.

Tabel 2.1 Threshold Core Web Vitals

Metrik	Aspek yang Diukur	Good	Needs Improvement	Poor
<i>Largest Contentful Paint</i> (LCP)	Kecepatan pemuatan konten utama	$\leq 2,5$ detik	$> 2,5-4$ detik	> 4 detik
<i>Interaction to Next Paint</i> (INP)	Responsivitas terhadap interaksi pengguna	≤ 200 ms	$> 200-500$ ms	> 500 ms
<i>Cumulative Layout Shift</i> (CLS)	Stabilitas visual halaman	$\leq 0,1$	$> 0,1-0,25$	$> 0,25$

2.6 Efisiensi Sumber Daya

Efisiensi sumber daya berkaitan dengan seberapa besar resource yang dibutuhkan oleh aplikasi web untuk dimuat dan dijalankan di browser. Aspek ini dapat dianalisis melalui JavaScript bundle size, total transferred size, dan request count. Metrik tersebut penting karena ukuran resource yang besar dapat memengaruhi waktu muat halaman, terutama pada tampilan mobile dan kondisi jaringan terbatas.

2.7 ElysiaJS dan Eden Treaty

ElysiaJS merupakan framework backend berbasis Bun dan TypeScript yang digunakan untuk membangun service API. Dalam penelitian ini, ElysiaJS digunakan sebagai backend yang sama untuk kedua frontend agar dataset, endpoint, dan response shape tetap konsisten. Eden Treaty digunakan sebagai type-safe API client agar komunikasi antara frontend dan backend memiliki kontrak tipe yang lebih aman dan konsisten.

2.8 Website Listing Properti

Website listing properti adalah website yang menampilkan daftar properti dalam bentuk katalog atau grid. Website jenis ini umumnya memiliki fitur pencarian, filter, sorting, pagination, dan halaman detail. Karakteristik tersebut cocok digunakan sebagai studi kasus untuk menguji performa frontend karena melibatkan pemuatan data visual, pengelolaan daftar data, serta interaksi pengguna.

2.8.1 Unit Testing

Unit testing adalah pengujian perangkat lunak yang dilakukan pada bagian kecil dari sistem, seperti fungsi, modul, atau komponen tertentu, secara terpisah dari keseluruhan aplikasi. Pengujian ini bertujuan untuk memastikan bahwa setiap bagian logic menghasilkan keluaran yang sesuai dengan input yang diberikan. Dalam penelitian ini, unit testing digunakan untuk memvalidasi kesetaraan logic pada implementasi VueJS dan Svelte, seperti pencarian, filter, sorting, pagination, dan formatting data properti.

2.8.2 Integration Testing

Integration testing adalah pengujian perangkat lunak yang dilakukan untuk memastikan bahwa beberapa modul atau komponen yang saling terhubung dapat bekerja bersama sesuai dengan kebutuhan sistem. Berbeda dengan unit testing yang berfokus pada bagian kecil secara terpisah, integration testing menguji hubungan antarbagian, seperti interaksi antara frontend dengan backend, pemrosesan data dari API, serta kesesuaian data yang ditampilkan pada antarmuka. Dalam penelitian ini, integration testing digunakan untuk memvalidasi bahwa prototipe VueJS dan Svelte dapat mengambil, mengolah, dan menampilkan data dari service backend ElysiaJS dengan hasil yang setara sebelum dilakukan pengujian performa.

2.8.3 End-to-End Testing

End-to-end testing adalah pengujian perangkat lunak yang dilakukan dengan mensimulasikan alur penggunaan aplikasi dari sisi pengguna secara menyeluruh. (Azizullah, Rahayudi and Priyambadha, 2026) menjelaskan bahwa end-to-end testing mensimulasikan skenario pengguna asli yang berinteraksi dengan aplikasi dari awal hingga akhir untuk memastikan komponen dan subsistem berjalan bersama sesuai harapan. Dalam penelitian ini, end-to-end testing digunakan untuk memvalidasi alur penggunaan pada kedua prototipe, seperti membuka halaman listing, melakukan pencarian, menerapkan filter, membuka halaman detail, dan menggunakan wishlist dummy.

2.8.4 Web Performance Testing

Web performance testing merupakan proses pengukuran terhadap performa aplikasi web untuk mengetahui bagaimana halaman dimuat, ditampilkan, dan merespons aktivitas pengguna. Pengukuran performa web dapat mencakup aspek seperti waktu pemuatan, proses rendering, responsivitas, penggunaan jaringan, serta penggunaan sumber daya komputasi pada browser. W3C melalui Web Performance Working Group mengembangkan berbagai mekanisme dan API yang memungkinkan pengukuran performa jaringan, rendering, responsivitas, interaktivitas, penggunaan memori, dan penggunaan CPU pada aplikasi web.

Pengujian performa web secara umum dapat dibedakan menjadi pengukuran lab dan field. Lab testing dilakukan pada lingkungan yang dikontrol dengan kondisi perangkat dan jaringan tertentu sehingga pengujian dapat dilakukan secara berulang dan konsisten. Sementara itu, field testing menggunakan data dari pengguna nyata dengan variasi perangkat, jaringan, dan kondisi penggunaan yang terjadi sebenarnya. Kedua pendekatan memiliki tujuan yang berbeda, dengan pengujian lab lebih sesuai untuk pengembangan, eksperimen, dan identifikasi regresi performa, sedangkan data field digunakan untuk mengetahui performa yang benar-benar dialami pengguna.

Salah satu alat yang dapat digunakan untuk melakukan pengujian performa pada lingkungan lab adalah Google Lighthouse. Lighthouse merupakan alat audit otomatis dan open-source yang dapat mengevaluasi sejumlah aspek kualitas halaman web, termasuk performa, aksesibilitas, dan SEO. Pada aspek performa, hasil Lighthouse dapat digunakan untuk memperoleh sejumlah metrik dan diagnosis terkait proses pemuatan serta rendering halaman.

2.8.5 User Experience

User Experience (UX) merupakan pengalaman yang muncul dari interaksi pengguna dengan suatu sistem, produk, atau layanan. Berdasarkan ISO 9241-210, pengalaman pengguna mencakup berbagai persepsi dan respons pengguna, seperti emosi, keyakinan, preferensi, persepsi, respons fisik dan psikologis, perilaku, serta pencapaian yang terjadi sebelum, selama, maupun setelah penggunaan suatu sistem. Dengan demikian, UX memiliki cakupan yang lebih luas daripada sekadar kemampuan pengguna dalam menyelesaikan suatu tugas.

Pengalaman pengguna pada aplikasi web dapat dipengaruhi oleh berbagai aspek antarmuka dan kualitas sistem. Selain desain visual dan kemudahan penggunaan, performa teknis juga dapat menjadi bagian yang mendukung pengalaman pengguna karena menentukan seberapa cepat konten dapat ditampilkan, seberapa responsif sistem terhadap tindakan pengguna, serta seberapa stabil tampilan ketika digunakan. Core Web Vitals sendiri dikembangkan untuk merepresentasikan aspek pengalaman pengguna pada web yang berkaitan dengan pemuatan, responsivitas, dan kestabilan visual.

2.8.6 Usability Testing

Usability berkaitan dengan kemampuan suatu sistem, produk, atau layanan untuk digunakan oleh pengguna dalam mencapai tujuan tertentu dalam konteks penggunaan yang ditentukan. ISO 9241-11 memberikan kerangka untuk memahami dan menerapkan konsep usability pada sistem interaktif. Aspek usability pada umumnya berkaitan dengan sejauh mana pengguna mampu menyelesaikan tujuan secara efektif, efisien, dan memperoleh tingkat kepuasan tertentu ketika menggunakan suatu sistem.

Usability testing merupakan metode evaluasi yang melibatkan pengguna untuk berinteraksi secara langsung dengan suatu antarmuka melalui serangkaian tugas yang merepresentasikan aktivitas penggunaan sebenarnya. Dalam pelaksanaannya, peserta diminta menjalankan tugas tertentu sementara peneliti mengamati perilaku, keberhasilan penyelesaian tugas, kesulitan yang muncul, serta umpan balik yang diberikan peserta. Tugas yang diberikan sebaiknya merepresentasikan tujuan pengguna sebenarnya agar temuan pengujian relevan terhadap kondisi penggunaan aplikasi.

2.8.7 System Usability Scale (SUS)

System Usability Scale (SUS) merupakan kuesioner terstandar yang dikembangkan oleh Brooke (1996) untuk mengukur tingkat kebolegunaan (*usability*) suatu sistem atau produk secara cepat, andal, dan efisien. SUS terdiri atas 10 item pertanyaan menggunakan skala Likert 5 poin, mulai dari angka 1 (*Sangat Tidak Setuju*) hingga 5 (*Sangat Setuju*). Item pertanyaan pada SUS disusun secara selang-seling antara pernyataan bernada positif pada item bernomor ganjil (1, 3, 5, 7, 9) dan pernyataan bernada negatif pada item bernomor genap (2, 4, 6, 8, 10).

Skor SUS dihitung dengan mengonversi nilai dari setiap item pertanyaan terlebih dahulu. Untuk item bernomor ganjil, kontribusi skor diperoleh dari nilai jawaban dikurangi 1. Sementara untuk item bernomor genap, kontribusi skor diperoleh dari 5 dikurangi nilai jawaban. Jumlah dari keseluruhan kontribusi skor 10 item tersebut kemudian dikalikan dengan 2,5 untuk menghasilkan total skor SUS dalam rentang 0 hingga 100.

Secara matematis, formula perhitungan skor SUS untuk satu responden dirumuskan sebagai berikut:

$$\text{Skor SUS} = 2,5 \times \left[\sum_{i \in \{1,3,5,7,9\}} (R_i - 1) + \sum_{i \in \{2,4,6,8,10\}} (5 - R_i) \right] \quad (2.1)$$

Keterangan:

- R_i = Nilai jawaban responden pada item pertanyaan ke- i (rentang 1–5).
- $(R_i - 1)$ = Kontribusi skor untuk item bernomor ganjil (pernyataan positif).
- $(5 - R_i)$ = Kontribusi skor untuk item bernomor genap (pernyataan negatif).
- 2,5 = Pengali konversi agar total skor berada pada rentang 0–100.

Hasil skor rata-rata SUS dari seluruh responden kemudian diinterpretasikan untuk menentukan tingkat keberterimaan (*acceptability*), tingkatan (*grade scale*), dan predikat (*adjective rating*) dari sistem yang dievaluasi (Bangor, Kortum and Miller, 2008). Skor SUS di atas 68 umumnya dikategorikan di atas rata-rata (*above average*) atau berada pada kategori dapat diterima (*acceptable*).

2.8.8 Vitest

Vitest adalah testing framework yang digunakan untuk menjalankan pengujian unit pada aplikasi berbasis JavaScript atau TypeScript. Vitest dipilih karena dapat digunakan pada ekosistem frontend modern dan mendukung proses pengujian logic secara cepat. Dalam penelitian ini, Vitest digunakan untuk menguji fungsi-fungsi pengolahan data pada prototipe VueJS dan Svelte agar logic yang digunakan pada kedua implementasi menghasilkan keluaran yang setara.

2.8.9 Playwright

Playwright adalah alat pengujian end-to-end yang digunakan untuk menjalankan skenario penggunaan aplikasi melalui browser secara otomatis. (Azizullah, Rahayudi and Priyambadha, 2026) menjelaskan bahwa Playwright mendukung otomatisasi pengujian end-to-end pada aplikasi web dan dapat berjalan pada beberapa browser utama seperti Chromium, Firefox, dan WebKit. Dengan kemampuan tersebut, Playwright digunakan dalam penelitian ini untuk memastikan fitur dan alur interaksi pada prototipe Vue.js and Svelte berjalan setara pada viewport mobile serta bertindak sebagai *interaction driver* pada pengujian performa responsivitas interaksi (INP).

2.8.10 Google Lighthouse

Google Lighthouse merupakan alat audit otomatis dan open-source yang digunakan untuk mengevaluasi kualitas halaman web. Lighthouse dapat dijalankan melalui Chrome DevTools, command line, maupun sebagai modul Node.js. Proses audit dilakukan dengan menjalankan serangkaian pengujian terhadap suatu halaman web kemudian menghasilkan laporan yang mencakup beberapa aspek, salah satunya performa halaman.

Pada aspek performa, Lighthouse memberikan sejumlah metrik, diagnostics, dan insights yang dapat digunakan untuk menganalisis proses pemuatan dan rendering halaman. Nilai Performance Score Lighthouse diperoleh dari kombinasi berbobot beberapa metrik performa. Namun, hasil pengukuran dapat mengalami variasi akibat kondisi

lingkungan pengujian sehingga pengujian berulang diperlukan untuk memperoleh hasil yang lebih representatif.

2.8.11 Chrome DevTools

Chrome DevTools merupakan sekumpulan alat pengembangan yang terintegrasi pada browser Google Chrome dan digunakan untuk melakukan inspeksi, debugging, serta analisis terhadap aplikasi web. Chrome DevTools menyediakan berbagai panel, seperti Elements, Network, Performance, dan Memory, yang dapat digunakan untuk mengamati struktur halaman, aktivitas jaringan, proses eksekusi aplikasi, serta penggunaan sumber daya pada browser.

Dalam analisis performa web, Network Panel dapat digunakan untuk merekam aktivitas permintaan jaringan ketika halaman dimuat. Informasi yang tersedia pada panel ini mencakup sumber daya yang diminta oleh halaman, ukuran data yang ditransfer, jumlah permintaan, waktu pemuatan, serta jenis sumber daya yang digunakan. Data tersebut dapat digunakan untuk mengetahui seberapa besar sumber daya yang diperlukan oleh suatu aplikasi ketika dijalankan pada browser.

Selain itu, Performance Panel dapat digunakan untuk merekam dan menganalisis aktivitas aplikasi selama proses pemuatan halaman maupun ketika pengguna melakukan interaksi. Panel ini menyediakan informasi mengenai aktivitas main thread, eksekusi JavaScript, proses scripting, rendering, dan layout. Informasi tersebut dapat digunakan untuk mengidentifikasi aktivitas komputasi yang berpotensi menjadi bottleneck pada aplikasi web.

BAB 3

METODOLOGI PENELITIAN

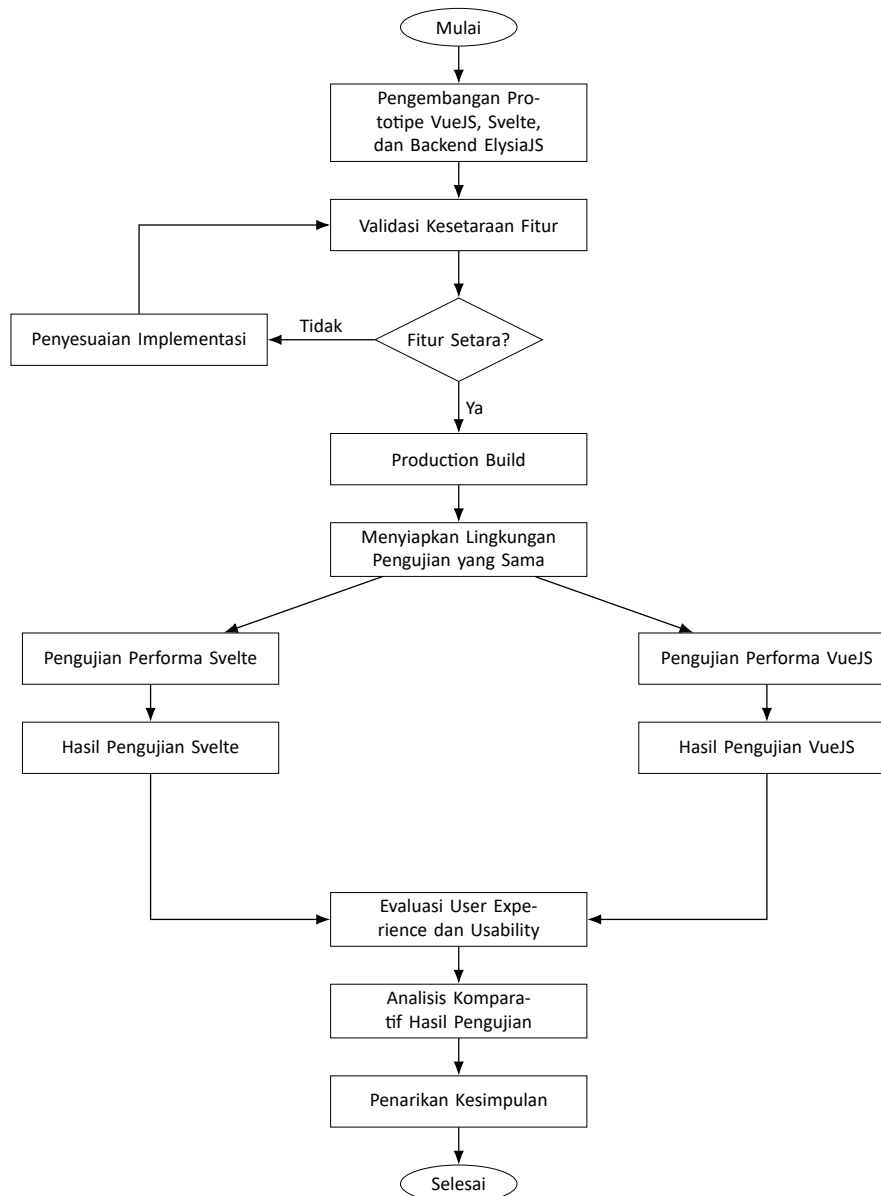
3.1 Rancangan Penelitian

Penelitian ini merupakan penelitian implementatif dengan pendekatan eksperimen komparatif. Dua prototipe website listing properti dikembangkan secara paralel menggunakan framework frontend VueJS dan Svelte dengan fitur, desain, dataset, aset, dan service backend yang dibuat setara. Setelah kesetaraan divalidasi, pengujian performa komputasional dan evaluasi pengalaman pengguna dilakukan terhadap kedua prototipe pada kondisi yang sama.

Alur umum penelitian terdiri atas delapan tahap berikut:

1. Pengembangan dua prototipe frontend (VueJS dan Svelte) beserta service backend ElysiaJS.
2. Validasi kesetaraan fitur dan perilaku kedua prototipe.
3. Pembuatan *production build* pada kedua prototipe.
4. Pengujian performa komputasional (Core Web Vitals, efisiensi sumber daya, dan respons interaksi).
5. Evaluasi *User Experience* menggunakan UEQ dan *usability* menggunakan SUS.
6. Analisis data hasil pengujian dan evaluasi.
7. Analisis komparatif hasil pengujian.
8. Penarikan kesimpulan.

Flowchart tahapan penelitian ditunjukkan pada Gambar 3.1.



Gambar 3.1 Flowchart Tahapan Penelitian

3.2 Persiapan dan Pengembangan Prototipe

Tahap persiapan dan pengembangan dilakukan dengan menyiapkan tiga bagian utama, yaitu frontend VueJS, frontend Svelte, dan service backend ElysiaJS. Kedua frontend dikembangkan sebagai prototipe website responsif listing properti dengan fitur yang sama, yaitu halaman daftar properti, halaman detail properti, pencarian, *advanced filter*, sorting, pagination atau *load more*, dan wishlist sederhana. Backend ElysiaJS digunakan sebagai sumber data yang sama agar kedua frontend mengakses endpoint, dataset, dan struktur response yang setara. Eden Treaty digunakan sebagai *type-safe API client* agar kontrak tipe antara frontend dan backend terjaga secara konsisten.

Pada tahap ini, tampilan kedua frontend disesuaikan agar memiliki layout, stylesheet, konten, aset, dan alur penggunaan yang sebanding, terutama pada viewport mobile. Prinsip yang diterapkan adalah *functional equivalence*, bukan *implementation equivalence*: kode VueJS dan Svelte tidak harus identik, tetapi perilaku yang terlihat oleh peng-

guna harus setara.

Variabel kontrol yang diterapkan pada kedua prototipe ditunjukkan pada Tabel 3.1.

Tabel 3.1 Variabel Kontrol Penelitian

Variabel Kontrol	Ketentuan
Backend	Service backend ElysiaJS yang sama
Dataset	Dataset properti yang sama
Aset	File gambar dan sumber daya yang sama
Desain	Layout dan stylesheet yang sama
Fitur	Fungsionalitas yang setara
Browser	Browser dan versi yang sama
Viewport	Ukuran viewport yang sama
Skenario	Interaksi pengujian yang sama

Dengan persiapan tersebut, perbandingan yang dilakukan dapat difokuskan pada perbedaan karakteristik VueJS dan Svelte sebagai framework frontend, bukan pada perbedaan data, fitur, desain, atau backend yang digunakan.

3.3 Validasi Kesetaraan Fitur

Validasi kesetaraan fitur dilakukan sebelum pengujian performa untuk memastikan bahwa implementasi VueJS dan Svelte memiliki fungsi, perilaku antarmuka, dan logika bisnis yang ekuivalen secara observabel. Validasi dibagi menjadi tiga tingkatan pengujian.

3.3.1 Unit Testing

Vitest digunakan untuk menguji logika pengolahan data secara terpisah pada masing-masing prototipe. Logika yang diuji meliputi pencarian, penyaringan, pengurutan, pagination, dan formatting data properti.

Untuk menjamin kesetaraan logika bisnis antar-framework, algoritme inti seperti pencarian dan penyaringan properti dibuat ekuivalen pada kedua implementasi. Algoritme penyaringan properti dirumuskan dalam bentuk pseudocode pada Kode 3.1.

Listing 3.1 Pseudocode Algoritme Penyaringan Properti Ekuivalen

```
1 function filterProperties(properties, query, filters):  
2   hasil = properties  
3  
4   if query tidak kosong:  
5     hasil = filter berdasarkan nama atau lokasi  
6  
7   if priceRange tersedia:  
8     hasil = filter berdasarkan rentang harga  
9  
10  if propertyType tersedia:  
11    hasil = filter berdasarkan tipe properti  
12  
13  if amenities tersedia:  
14    hasil = filter berdasarkan fasilitas (AND)  
15  
16  if guests tersedia:  
17    hasil = filter berdasarkan kapasitas minimum  
18  
19  return hasil
```

Penerapan algoritme ekuivalen ini memastikan bahwa logika pemrosesan data bisnis yang dipakai pada VueJS dan Svelte dibuat identik, sedangkan perbedaan hanya berada pada mekanisme implementasi internal framework.

3.3.2 Integration Testing

Validasi integrasi dilakukan untuk memastikan bahwa kedua prototipe frontend berinteraksi dengan service backend ElysiaJS secara setara. Aspek yang divalidasi meliputi:

- *Request* API ke endpoint yang benar;
- *Response* data yang diterima sesuai kontrak;
- Pemrosesan data dari API;
- Hasil data yang ditampilkan pada antarmuka.

3.3.3 End-to-End Testing

Playwright digunakan untuk memastikan alur penggunaan (*user flow*) pada prototipe VueJS dan Svelte berjalan setara pada viewport mobile. Skenario pengujian kesetaraan fitur ditunjukkan pada Tabel 3.2.

Tabel 3.2 Skenario Pengujian Kesetaraan Fitur

ID	Skenario	Hasil yang Diharapkan
F01	Search properti	Kedua prototipe menghasilkan data yang setara
F02	Advanced filter	Hasil filter setara
F03	Sorting	Urutan data setara
F04	Wishlist	Perubahan status wishlist berjalan setara
F05	Detail properti	Informasi properti yang ditampilkan setara

Apabila ditemukan perbedaan hasil atau perilaku, implementasi disesuaikan terlebih dahulu hingga kedua frontend memenuhi seluruh kriteria penerimaan.

3.4 Pengujian Performa Komputasional

Pengujian performa komputasional dilakukan setelah implementasi VueJS dan Svelte dinyatakan memiliki fitur dan perilaku yang setara. Kedua aplikasi terlebih dahulu dijalankan dalam mode *production build* agar pengukuran dilakukan terhadap versi aplikasi yang telah melalui proses optimasi build.

3.4.1 Lingkungan Pengujian

Seluruh pengujian performa komputasional dilakukan pada lingkungan yang dikontrol agar hasil pengukuran dapat dibandingkan secara adil. Konfigurasi lingkungan pengujian ditunjukkan pada Tabel 3.3.

Tabel 3.3 Konfigurasi Lingkungan Pengujian

Parameter	Konfigurasi
Perangkat keras	AMD Ryzen 7 4800H (8 Cores, 16 Threads @ 2.9 GHz, up to 4.2 GHz), RAM 16 GB DDR4, NVIDIA GeForce RTX 3050 Laptop GPU
Sistem operasi	Windows 11 64-bit
Browser	Google Chrome 151.0.7922.138
Infrastruktur backend	Cloudflare Workers (ElysiaJS REST API)
Infrastruktur database	Cloudflare D1 (Serverless SQLite dengan Drizzle ORM dan dataset deterministik)
API client contract	Eden Treaty (Type-safe client)
Viewport	390 × 844 piksel
CPU throttling	4 × slowdown
Network throttling	<i>Controlled network</i> / Fresh context
Cache policy	<i>Fresh browser context</i> pada setiap observasi
Build mode	Production build
Jumlah pengulangan	n kali per skenario per framework
Versi Lighthouse	Lighthouse CI 0.15.1
Versi Playwright	Playwright 1.62.1

Pengujian dilakukan pada kondisi perangkat, browser, viewport, dataset, aset, serta skenario penggunaan yang sama sehingga perbedaan hasil pengukuran dapat lebih difokuskan pada karakteristik implementasi masing-masing framework. Penempatan backend pada Cloudflare Workers dan basis data pada Cloudflare D1 memastikan bahwa pemrosesan query dan data yang dikirimkan ke frontend berada pada kondisi lingkungan komputasi edge yang identik serta terisolasi dari beban komputasi mesin pengujian.

3.4.2 Pengujian LCP dan CLS

Google Lighthouse digunakan untuk mengukur performa pemuatan dan stabilitas tampilan kedua prototipe pada viewport perangkat bergerak dalam mode *production build*. Metrik utama yang digunakan dalam konteks Core Web Vitals adalah Largest Contentful Paint (LCP) dan Cumulative Layout Shift (CLS). Selain itu, beberapa metrik diagnostik Lighthouse seperti First Contentful Paint (FCP), Total Blocking Time (TBT), Speed Index, dan Performance Score digunakan sebagai data pendukung dalam proses analisis. Metrik diagnostik tersebut bukan merupakan bagian dari Core Web Vitals.

Prosedur pengujian dilakukan dengan alur berikut:

1. Membangun *production build* pada kedua prototipe.
2. Menjalankan aplikasi pada lingkungan pengujian yang telah dikonfigurasi.
3. Menyiapkan kondisi pengujian yang sama (browser, viewport, cache policy).
4. Menjalankan Google Lighthouse pada masing-masing prototipe.
5. Menyimpan hasil metrik dari setiap pengujian.
6. Mengulangi pengujian sebanyak n kali.
7. Menghitung nilai representatif (median) dari seluruh pengulangan.

3.4.3 Pengujian Interaction to Next Paint

Pengukuran Interaction to Next Paint (INP) dilakukan melalui skenario interaksi yang diotomatisasi menggunakan Playwright. Otomasi digunakan untuk mengurangi variasi yang dapat muncul apabila interaksi dilakukan secara manual serta memastikan bahwa kedua implementasi memperoleh prosedur pengujian yang setara.

Playwright bertindak sebagai *interaction driver* untuk mensimulasikan interaksi pengguna, sedangkan nilai INP itu sendiri diukur secara langsung oleh *Event Timing API* browser Chrome. Prosedur otomasi dan perekaman durasi interaksi dirumuskan dalam bentuk pseudocode pada Kode 3.2.

Listing 3.2 Pseudocode Algoritme Pengukuran INP

```
1  FOR setiap framework
2    FOR setiap skenario
3      FOR setiap run
4        Buat browser context baru
5        Atur viewport 390 x 844
6        Terapkan CPU throttling 4x
7        Pasang PerformanceObserver (Event Timing)
8        Buka production build
9        Jalankan interaksi menggunakan Playwright
10       Rekam interaction timing
11       Simpan nilai INP
12     END FOR
13
14     Hitung median
15     Hitung IQR
16   END FOR
17 END FOR
```

Pengukuran dilakukan dengan menyuntikkan *PerformanceObserver* ke dalam konteks halaman browser. Potongan kode setup perekaman *Event Timing API* ditunjukkan pada Kode 3.3.

Listing 3.3 Setup PerformanceObserver untuk Merekam Event Timing API Browser

```
1  const observer = new PerformanceObserver((list) => {
2    for (const entry of list.getEntries()) {
3      if (entry.interactionId > 0) {
4        interactions.push({
5          interactionId: entry.interactionId,
6          duration: entry.duration
7        });
8      }
9    }
10  });
11
12  observer.observe({
13    type: "event",
14    buffered: true,
15    durationThreshold: 16
16  });
```

Pengujian dijalankan terhadap tiga skenario interaksi utama. Salah satu contoh potongan skenario otomatisasi Playwright pada fitur *advanced filter* ditunjukkan pada Kode 3.4.

Listing 3.4 Potongan Skenario Otomatisasi Playwright pada Advanced Filter

```
1  await page.getByLabel("Lokasi").selectOption({ label: "Senggigi, Lombok" });
2  await page.getByLabel("Tipe").selectOption({ label: "Villa" });
3  await page.getByText("Filter lanjutan", { exact: true }).click();
4  await page.getByLabel("Tamu minimum").fill("6");
5  await startMeasurement(page);
6  await page.getByLabel("Kitchen").check();
7  const measurement = await finishMeasurement(page);
```

Skenario otomatisasi yang sama dijalankan secara ekuivalen terhadap prototipe VueJS dan Svelte. Ringkasan seluruh skenario interaksi yang diuji ditunjukkan pada Tabel 3.4.

Tabel 3.4 Skenario Interaksi Pengukuran INP

Skenario	Interaksi Pengguna
Search	Membuka pencarian dan menerapkan kata kunci lokasi atau nama properti
Advanced Filter	Membuka filter lanjutan, memilih parameter tipe, tamu, dan fasilitas, lalu menerapkannya
Wishlist	Mengubah status wishlist properti (toggle simpan atau hapus)

3.4.4 Pengujian Efisiensi Sumber Daya

Pengujian efisiensi sumber daya dilakukan menggunakan hasil *production build* dan Chrome DevTools, khususnya melalui Network Panel. Metrik yang diamati meliputi:

- **JavaScript bundle size:** ukuran kode JavaScript yang perlu dikirim dan diproses oleh browser.
- **Total transferred size:** keseluruhan data yang ditransfer melalui jaringan ketika halaman dimuat.
- **Request count:** jumlah permintaan sumber daya yang dilakukan oleh browser.

Seluruh pengukuran dilakukan pada halaman, dataset, aset, browser, dan kondisi jaringan yang sama. Dengan demikian, perbandingan efisiensi sumber daya dapat digunakan sebagai data pendukung untuk menjelaskan perbedaan karakteristik performa komputasional antara VueJS dan Svelte.

3.5 Evaluasi User Experience dan Usability

3.5.1 Partisipan Penelitian

Evaluasi *User Experience* dan *usability* melibatkan responden yang mewakili target pengguna prototipe website listing properti. Kriteria inklusi meliputi pengguna yang memiliki pengalaman mengakses website melalui perangkat bergerak. Metode pengambilan sampel dan jumlah responden akan ditentukan sebelum pelaksanaan evaluasi.

3.5.2 Skenario Pengujian

Responden diminta menjalankan rangkaian tugas yang sama pada prototipe VueJS dan Svelte. Tugas yang diberikan meliputi:

1. Mencari properti berdasarkan kata kunci.
2. Menggunakan *advanced filter* untuk menyaring properti.
3. Mengurutkan daftar properti.
4. Membuka halaman detail properti.
5. Menambahkan properti ke wishlist.

3.5.3 Prosedur Pengujian

Pengujian menggunakan desain *within-subject*: setiap responden mencoba kedua prototipe. Untuk mengurangi *order effect*, diterapkan *counterbalancing* sebagai berikut:

- **Kelompok A:** VueJS terlebih dahulu, kemudian Svelte.
- **Kelompok B:** Svelte terlebih dahulu, kemudian VueJS.

Setelah menggunakan masing-masing prototipe, responden mengisi kuesioner User Experience Questionnaire (UEQ) dan System Usability Scale (SUS). Skenario, data, tampilan, dan kondisi penggunaan dibuat setara pada kedua prototipe.

3.5.4 User Experience Questionnaire

User Experience Questionnaire (UEQ) diberikan kepada responden setelah menyelesaikan rangkaian tugas pada masing-masing prototipe. Hasil UEQ dihitung pada enam skala, yaitu *Attractiveness*, *Perspicuity*, *Efficiency*, *Dependability*, *Stimulation*, dan *Novelty*. Nilai UEQ pada prototipe VueJS dan Svelte kemudian dibandingkan.

3.5.5 System Usability Scale

System Usability Scale (SUS) diberikan setelah responden menggunakan masing-masing prototipe. SUS terdiri atas 10 item dengan skala Likert 5 poin. Skor akhir berada pada rentang 0–100. Perhitungan skor SUS menggunakan formula yang telah dijelaskan pada Bab 2. Skor SUS pada prototipe VueJS dan Svelte kemudian dibandingkan.

3.6 Teknik Analisis Data

3.6.1 Analisis Performa Komputasional

Hasil pengujian berulang dianalisis menggunakan statistik deskriptif. Nilai representatif yang digunakan meliputi median, minimum, maksimum, dan *interquartile range* (IQR).

$$IQR = Q_3 - Q_1 \quad (3.1)$$

Median digunakan karena hasil pengujian performa pada browser dapat mengalami variasi antar-run dan median lebih tahan terhadap nilai ekstrem dibandingkan satu hasil pengukuran tunggal.

Hasil pengukuran LCP, INP, dan CLS juga dibandingkan terhadap *threshold* Core Web Vitals yang ditunjukkan pada Tabel 3.5.

Tabel 3.5 Threshold Core Web Vitals

Metrik	Good	Needs Improvement	Poor	
LCP	$\leq 2,5$ detik	$> 2,5\text{--}4$ detik	> 4 detik	
INP	≤ 200 ms	$> 200\text{--}500$ ms	> 500 ms	
CLS	$\leq 0,1$	$> 0,1\text{--}0,25$	$> 0,25$	

Ringkasan aspek, metrik, dan instrumen pengukuran yang digunakan pada penelitian ini ditunjukkan pada Tabel 3.6.

Tabel 3.6 Instrumen dan Alat Pengukuran

Aspek	Alat / Metode
LCP dan CLS	Google Lighthouse
INP	Playwright (otomasi interaksi) dan Event Timing API browser (pengukuran INP)
JavaScript bundle size	Hasil production build dan Chrome DevTools
Total transferred size	Chrome DevTools Network
Request count	Chrome DevTools Network
User Experience	User Experience Questionnaire (UEQ)
Usability	System Usability Scale (SUS)
Kesetaraan fungsi	Vitest dan Playwright

3.6.2 Analisis UEQ

Hasil UEQ dihitung pada enam skala, yaitu *Attractiveness*, *Perspicuity*, *Efficiency*, *Dependability*, *Stimulation*, dan *Novelty*. Nilai pada masing-masing skala untuk prototipe VueJS dan Svelte kemudian dibandingkan untuk mengidentifikasi perbedaan pengalaman pengguna.

3.6.3 Analisis SUS

Skor SUS setiap responden dihitung menggunakan aturan:

- Item ganjil: nilai jawaban -1 .
- Item genap: $5 -$ nilai jawaban.
- Jumlah kontribusi seluruh item $\times 2,5$.

Skor SUS pada prototipe VueJS dan Svelte kemudian dibandingkan. Interpretasi skor mengacu pada kategori *acceptability*, *grade scale*, dan *adjective rating* sebagaimana dijelaskan pada Bab 2.

3.7 Analisis Komparatif Hasil Pengujian

Tahap analisis komparatif dilakukan untuk membandingkan hasil dari seluruh aspek pengujian secara keseluruhan guna menjawab rumusan masalah penelitian. Aspek yang dibandingkan meliputi:

- Core Web Vitals (LCP, INP, CLS);
- efisiensi sumber daya (JavaScript bundle size, total transferred size, request count);
- responsivitas interaksi (hasil INP pada skenario search, advanced filter, dan wishlist);
- *User Experience* (hasil UEQ pada enam skala);
- *usability* (skor SUS).

Hasil perbandingan tersebut digunakan untuk menganalisis bagaimana perbedaan performa komputasional, pengalaman pengguna, dan *usability* antara VueJS dan Svelte dapat menjadi pertimbangan dalam pemilihan framework frontend untuk pengembangan website listing yang kaya konten visual dan memiliki fitur interaktif.

DAFTAR REFERENSI

Andriyanai, F., Putra, R.R.P., Adnandi, M.A. and Maulana, I., 2025. OPTIMALISASI SEO BERBASIS CORE WEB VITALS PADA WEBSITE SMKN 1 KABUPATEN TANGERANG. *Journal Of Information Technology and Computer Science*, 2(2).

Azizullah, A., Rahayudi, B. and Priyambadha, B., 2026. Rancang Bangun Sistem Otomatisasi Pengujian Berbasis REST API Terintegrasi Dengan Playwright Pada PT XYZ.

Bangor, A., Kortum, P.T. and Miller, J.T., 2008. An empirical evaluation of the System Usability Scale. *International Journal of Human-Computer Interaction*, 24(6), pp.574–594. <https://doi.org/10.1080/10447310802205776>.

Brooke, J., 1996. SUS-A quick and dirty usability scale. *Usability evaluation in industry*, 189(194), pp.4–7.

Ferreira, F., Borges, H.S. and Valente, M.T., 2022. On the (un-)adoption of JavaScript front-end frameworks. *Software: Practice and Experience*, 52(4), pp.947–966. <https://doi.org/10.1002/spe.3044>.

Ollila, R., Mäkitalo, N. and Mikkonen, T., 2022. Modern Web Frameworks: A Comparison of Rendering Performance. *Journal of Web Engineering*. [online] <https://doi.org/10.13052/jwe1540-9589.21311>.

Putra, F.P.E., Hasbullah, H., Muslim, F. and Paradina, R., 2025. Technical Performance Comparison of Modern Frontend Frameworks Study on Svelte, React, and Vue. *Brilliance: Research of Artificial Intelligence*, 5(1), pp.355–364. <https://doi.org/10.47709/brilliance.v5i1.6133>.

Sofi'ie, F.A.F. and Qoiriah, A., 2023. Analisis Perbandingan Framework Front-End Javascript React dan Vue Pada Pengembangan Website. *Journal of Informatics and Computer Science (JINACS)*, 5(02), pp.157–164. <https://doi.org/10.26740/jinacs.v5n02.p157-164>.

<https://elysiajs.com/eden/treaty/overview.html#eden-treaty>

<https://vitest.dev/guide/>

<https://playwright.dev/docs/intro>